基于chinook.db 数据集实现问答agent

In [2]:
```
import sqlite3
import os
from typing import Dict, List, Any
from openai import OpenAI
import re

# 数据库连接与模式分析模块
class DatabaseAnalyzer:
    def __init__(self, db_path: str = "04_SQL-Code-Agent-Demo/chinook.db"):
        self.db_path = db_path
        self.conn = None
        self.schema_info = {}

    def connect(self):
        """连接到 SQLite 数据库"""
        self.conn = sqlite3.connect(self.db_path)
        self.conn.row_factory = sqlite3.Row

    def get_table_count(self) -> int:
        """获取数据库中表的总数"""
        cursor = self.conn.cursor()
        cursor.execute("""
            SELECT COUNT(*) as count
            FROM sqlite_master
            WHERE type='table' AND name NOT LIKE 'sqlite_%'
        """)
        return cursor.fetchone()['count']

    def get_table_schema(self) -> Dict:
        """获取所有表的详细结构信息"""
        cursor = self.conn.cursor()
        cursor.execute("""
            SELECT name, sql
            FROM sqlite_master
            WHERE type='table' AND name NOT LIKE 'sqlite_%'
        """)

        schema = {}
        for row in cursor.fetchall():
            table_name = row['name']
            schema[table_name] = self._parse_create_sql(row['sql'])

        return schema

    def _parse_create_sql(self, sql: str) -> Dict:
        """解析 CREATE TABLE SQL 语句"""
        # 简化解析，实际应用中可以使用更复杂的解析器
        return {"sql": sql}

    def get_record_count(self, table_name: str) -> int:
        """获取指定表的记录数"""
        cursor = self.conn.cursor()
        cursor.execute(f"SELECT COUNT(*) as count FROM {table_name}")
        return cursor.fetchone()['count']
```

```python
    def close(self):
        """关闭数据库连接"""
        if self.conn:
            self.conn.close()

# 自然语言到 SQL 转换模块
class NL2SQLConverter:
    def __init__(self, api_key: str,base_url:None,model_name:str):
        self.model_name = model_name
        self.client = OpenAI(api_key=api_key,base_url=base_url)
        self.system_prompt = """你是一个专业的 SQL 查询生成助手。根据用户的问题和

数据库结构信息：
{table_schema}

规则：
1. 只返回 SQL 查询语句，不要包含解释
2. 使用正确的表名和列名
3. 确保查询语法正确
4. 对于计数查询，使用 COUNT(*) 或 COUNT(column_name)
"""

    def generate_sql(self, question: str, table_schema: Dict) -> str:
        """根据自然语言问题生成 SQL 查询"""

        # 格式化数据库结构信息
        schema_text = "\n".join([f"表名: {table}\n创建语句: {info['sql']}"
                                 for table, info in table_schema.items()])

        prompt = self.system_prompt.format(table_schema=schema_text)

        try:
            response = self.client.chat.completions.create(
                model=self.model_name,
                messages=[
                    {"role": "system", "content": prompt},
                    {"role": "user", "content": question}
                ],
                temperature=0.1,
                max_tokens=200
            )

            sql = response.choices[0].message.content.strip()

            # 清理 SQL 语句（移除可能的代码块标记）
            sql = re.sub(r'```sql\n?|\n?```', '', sql)

            return sql

        except Exception as e:
            raise Exception(f"生成 SQL 失败: {str(e)}")
# 3. 查询执行与结果处理模块

class QueryExecutor:
    def __init__(self, db_analyzer: DatabaseAnalyzer):
        self.db_analyzer = db_analyzer

    def execute_query(self, sql: str) -> Any:
        """执行 SQL 查询并返回结果"""
        cursor = self.db_analyzer.conn.cursor()
```

```python
        try:
            cursor.execute(sql)

            # 获取结果
            if sql.strip().upper().startswith("SELECT"):
                results = cursor.fetchall()

                # 转换为字典列表
                columns = [description[0] for description in cursor.description]
                return [dict(zip(columns, row)) for row in results]
            else:
                self.db_analyzer.conn.commit()
                return {"affected_rows": cursor.rowcount}

        except sqlite3.Error as e:
            raise Exception(f"SQL 执行错误: {str(e)}")

# 主agent类
class ChinookQAAgent:
    def __init__(self, openai_api_key: str,
                 base_url:str,
                 model_name:str='qwen3.6-plus',
                 db_path: str = "04_SQL-Code-Agent-Demo/chinook.db"):
        self.db_analyzer = DatabaseAnalyzer(db_path)
        self.nl2sql_converter = NL2SQLConverter(api_key=openai_api_key,
                                                base_url=base_url,
                                                model_name=model_name)
        self.query_executor = None

    def initialize(self):
        """初始化 Agent"""
        print("🔧 正在初始化 Chinook 数据库问答 Agent...")

        # 连接数据库
        self.db_analyzer.connect()

        # 获取数据库结构
        schema = self.db_analyzer.get_table_schema()
        print(f"✅ 已连接到数据库，发现 {len(schema)} 张表")

        # 初始化查询执行器
        self.query_executor = QueryExecutor(self.db_analyzer)

        print("🚀 Agent 初始化完成，可以开始提问！")

    def ask_question(self, question: str) -> Dict:
        """回答用户问题"""
        print(f"\n📝 用户提问: {question}")

        try:
            # 1. 获取数据库结构
            schema = self.db_analyzer.get_table_schema()

            # 2. 生成 SQL 查询
            print("🤖 正在生成 SQL 查询...")
            sql = self.nl2sql_converter.generate_sql(question, schema)
            print(f"📊 生成的 SQL: {sql}")

            # 3. 执行查询
```

```python
            print("⚡ 正在执行查询...")
            result = self.query_executor.execute_query(sql)

            # 4. 格式化结果
            response = {
                "question": question,
                "sql_query": sql,
                "result": result,
                "status": "success"
            }

            print(f"✅ 查询成功完成")
            return response

        except Exception as e:
            return {
                "question": question,
                "error": str(e),
                "status": "error"
            }

    def close(self):
        """关闭 Agent"""
        self.db_analyzer.close()
        print("👋 Agent 已关闭")


def main():
    # 配置
    OPENAI_API_KEY = os.getenv('QWEN_API_KEY')

    # 创建 Agent
    agent = ChinookQAAgent(
        openai_api_key=OPENAI_API_KEY,
        base_url='https://dashscope.aliyuncs.com/compatible-mode/v1',
        db_path="Week12/04_SQL-Code-Agent-Demo/chinook.db"
    )

    # 初始化
    agent.initialize()

    # 测试问题
    test_questions = [
        "数据库中总共有多少张表",
        "员工表中有多少条记录",
        "在数据库中所有客户个数和员工个数分别是多少"
    ]

    # 回答问题
    for question in test_questions:
        response = agent.ask_question(question)

        print("\n" + "=" * 50)
        print(f"📋 问题: {response['question']}")
        print(f"🔍 SQL: {response['sql_query']}")

        if response['status'] == 'success':
            print(f"📊 结果: {response['result']}")
        else:
            print(f"❌ 错误: {response['error']}")
```

```
        print("=" * 50 + "\n")

    # 关闭 Agent
    agent.close()


if __name__ == "__main__":
    main()
```

```
🔧 正在初始化 Chinook 数据库问答 Agent...
✅ 已连接到数据库，发现 11 张表
🚀 Agent 初始化完成，可以开始提问！

📝 用户提问: 数据库中总共有多少张表
🤖 正在生成 SQL 查询...
📊 生成的 SQL: SELECT COUNT(*) FROM sqlite_master WHERE type='table';
⚡ 正在执行查询...
✅ 查询成功完成

==================================================
📋 问题: 数据库中总共有多少张表
🔍 SQL: SELECT COUNT(*) FROM sqlite_master WHERE type='table';
📊 结果: [{'COUNT(*)': 13}]
==================================================


📝 用户提问: 员工表中有多少条记录
🤖 正在生成 SQL 查询...
📊 生成的 SQL: SELECT COUNT(*) FROM employees;
⚡ 正在执行查询...
✅ 查询成功完成

==================================================
📋 问题: 员工表中有多少条记录
🔍 SQL: SELECT COUNT(*) FROM employees;
📊 结果: [{'COUNT(*)': 8}]
==================================================


📝 用户提问: 在数据库中所有客户个数和员工个数分别是多少
🤖 正在生成 SQL 查询...
📊 生成的 SQL: SELECT (SELECT COUNT(*) FROM customers) AS CustomerCount, (SELECT
COUNT(*) FROM employees) AS EmployeeCount;
⚡ 正在执行查询...
✅ 查询成功完成

==================================================
📋 问题: 在数据库中所有客户个数和员工个数分别是多少
🔍 SQL: SELECT (SELECT COUNT(*) FROM customers) AS CustomerCount, (SELECT COUNT
(*) FROM employees) AS EmployeeCount;
📊 结果: [{'CustomerCount': 59, 'EmployeeCount': 8}]
==================================================

👋 Agent 已关闭
```

阅读 06-stock-bi-agent 代码，回答两个问题

1. 什么是前后端分离?

前段负责界面交互，后端负责业务处理，以这个项目为例： 前端：‘streamlit.py’、‘chat.py’等Streamlit页面。负责界面展示，用户输入，历史对话展示，调用后端接口等功能。 后端：‘main_server.py’启动FastAPI服务，并挂载chat.py,user.py,data.py,stock.py等接口，负责业务逻辑，数据库读写，调用大模型与工具

2. 历史对话如何存储?

这个项目有两套历史对话存储功能: 面，系统将会话元信息和消息内容分别存储在 SQLite 数据库 sever.db 的 chat_session 表与 chat_message 表中，用于前端展示、历史查询和反馈管理；另一方面，系统通过 AdvancedSQLiteSession 将 Agent 运行时会话状态存储到 conversations.db 中，用于维护大模型多轮对话上下文。

3、历史对话如何作为下一次输入

当用户继续在同一 session_id 下发起新问题时，后端会基于该 session_id 创建 AdvancedSQLiteSession，并在 Runner.run_streamed(agent, input=content, session=session) 调用中自动读取该会话在 conversations.db 中保存的历史消息。这样，历史对话无需由前端显式传回，而是由会话管理组件自动恢复，并与当前用户输入共同组成下一次大模型推理的上下文。